markovlib & markovlv

Discrete-Time Markov Models for Life Insurance

Technical Reference and User Guide

Michael Koller

Original C++ implementation, 2011 / 2026

Python translation and documentation

April 2026

Version: markovlib.py v2.0 (corrected Python translation)
Version: markovlv.py v2.7.6 (compiled C++ version via pypi)
Standard library only. No external dependencies.

Reference: M. Koller, *Stochastische Modelle in der Lebensversicherung*, Springer, 2000 (2nd ed. 2012). Cited throughout as **[Koller]**.

Contents

1 Introduction **3**

2 Theory: Discrete-Time Markov Models in Life Insurance **3**
2.1 The State Process . . . 3
2.2 Chapman–Kolmogorov Equations . . . 4
2.3 Cash Flows . . . 4
2.4 Thiele's Difference Equation . . . 4
2.5 Technical Premium Decomposition . . . 5
2.6 Mortality Projection . . . 5

3 Library Architecture **5**
3.1 Overview . . . 5
3.2 Configuration: `config.json` . . . 6
3.3 Mortality Tables: `tables.json` . . . 6

4 Module Reference **7**
4.1 `TABLESERVER` — Mortality Table Wrapper . . . 7
4.1.1 Purpose . . . 7
4.1.2 Constructor . . . 7
4.1.3 Methods . . . 7
4.1.4 Module-level functions . . . 8
4.2 `MARKOVLV` — Core Markov Engine . . . 8
4.2.1 Purpose . . . 8
4.2.2 Constructor . . . 8
4.2.3 Configuration setters . . . 8
4.2.4 Input setters . . . 8
4.2.5 Result getters . . . 9
4.2.6 Simulation . . . 9
4.3 `CAPITALLV` — Endowment / Term-Life . . . 9
4.3.1 Purpose . . . 9
4.3.2 State space . . . 9
4.3.3 Key methods . . . 10
4.4 `ANNUITYLV` — Single-Life Annuity . . . 10
4.4.1 Purpose . . . 10
4.4.2 State space . . . 10
4.4.3 Key methods . . . 11
4.5 `ANNUITYLV2` — Two-Life Annuity . . . 11
4.5.1 Purpose . . . 11
4.5.2 State space . . . 11
4.5.3 Key methods . . . 11
4.6 `WIDDOWLV` — Widow Pension . . . 12
4.6.1 Purpose . . . 12
4.6.2 State space . . . 12
4.6.3 Key methods . . . 12
4.7 `GLMOD` — Group Life Portfolio (402 states) . . . 12
4.7.1 Purpose . . . 12

4.7.2 State layout 13
4.7.3 Key methods 13
4.8 ANNMOD — Annuity Portfolio (1702 states) 14
4.8.1 Purpose 14
4.8.2 State layout 14
4.8.3 Key methods 14

5 Examples **15**
5.1 Example 1: Simple 2-State Model (Manual Setup) 15
5.2 Example 2: Single-Life Annuity from Table 15
5.3 Example 3: Endowment Contract 16
5.4 Example 4: Two-Life Last-Survivor Annuity 16
5.5 Example 5: Widow Pension 17
5.6 Example 7: Adding a Custom Mortality Table 17

A Appendix A: `config.json` — Example File **18**

B Appendix B: `tables.json` — Structure and Example **19**

C Test Results **20**

1 Introduction

`markovlib` is a pure-Python translation of the C++ actuarial library *omarkov*, originally developed by Michael Koller (2011, revised 2026). The latter one has the same interface and can be installed via pypi install markovlv. In case one wants to use this replace import markovlib by import markovlv. It provides a complete toolkit for pricing and reserving life-insurance products modelled as discrete-time Markov chains. The library uses only the Python standard library; no third-party packages are required.

The library covers:

- A generic multi-state Markov engine (`MARKOVLV`) implementing Thiele's difference equation, Chapman–Kolmogorov forward recursion, and technical premium decomposition.
- Special-purpose subclasses for the most common actuarial products: endowments (`CAPITALLV`), single-life annuities (`ANNUITYLV`), two-life annuities (`ANNUITYLV2`), and widow pensions (`WIDDOWLV`).
- Portfolio-level models aggregating many policies: group-life (`GLMOD`) and annuity module (`ANNMOD`).
- A mortality table server (`TABLESERVER`) backed by `tables.json` with 157 built-in actuarial tables.
- Runtime configuration via `config.json`.

All theory in this document follows the notation and treatment of **[Koller]**, to which the reader is referred for proofs and extended discussion.

Important disclaimer: The current python library is still in development and does not pass full regression test hence it is advisable to compare its output with markovlv, the C++ based fully tested library. Known shortcomings:

- Cash Flows for annuitylv when there are guarantees
- Collective Widows module WIDDOWLV (ie $\sum k = 0_k^\infty p_x\, q_{x+k}\, h_{x+k}\, v^{k+1}\, \ddot{a}_{y_{x+k}}$)
- Simulation is untested

Please refer to secrtion C for further details.

2 Theory: Discrete-Time Markov Models in Life Insurance

2.1 The State Process

Let time be measured in integers $t = 0, 1, 2, \ldots$. We model an insured life (or a group of lives) by a *finite-state Markov chain* $\{I_t\}_{t\geq 0}$ taking values in the state space $\mathcal{S} = \{0, 1, \ldots, n-1\}$.

Definition 2.1 (Markov property)**.** $\{I_t\}$ is a Markov chain if for all t and all states i, j,

$$\Pr(I_{t+1} = j \mid I_t = i, I_{t-1}, \ldots, I_0) = \Pr(I_{t+1} = j \mid I_t = i) =: p_{ij}(t).$$

The numbers $p_{ij}(t) \geq 0$ are the *one-step transition probabilities* at time t. They satisfy $\sum_j p_{ij}(t) = 1$ for every i. In insurance models typical states are:

- **0 = alive / healthy** (or *active*)
- **1 = dead** (absorbing)
- Additional states for disability, guarantee periods, etc.

2.2 Chapman–Kolmogorov Equations

The k-step transition probabilities are obtained by the *Chapman–Kolmogorov* equation:

$$p_{ij}^{(k)}(s,t) := \Pr(I_{X_{s+k}=j} \mid I_{X_s=i}) = \sum_{\ell \in \mathcal{S}} p_{i\ell}(s)\, p_{\ell j}^{(k-1)}(s+1,t), \qquad k \geq 1, \tag{1}$$

with $p_{ij}^{(0)}(s,s) = \mathbf{1}_{X_{i=j}}$. In matrix notation, with $\mathbf{P}(t) = (p_{ij}(t)) \in \mathcal{M}(\mathbb{R}^n)$:

$$\mathbf{P}^{(k)}(s, s+k) = \mathbf{P}(s)\,\mathbf{P}(s+1)\cdots\mathbf{P}(s+k-1),$$

where the multiplication in this context is the matrix multiplication in $\mathcal{M}(\mathbb{R}^n)$. The forward recursion propagates state distributions and is the basis of the cash-flow computation in `_compute_cf`.

2.3 Cash Flows

We distinguish two types of benefit payment:
Definition 2.2 (Pre- and post-transition benefits)**.**

- $a_i^{\text{pre}}(t)$: benefit paid at the *beginning* of period $[t, t+1)$ while in state i (before any transition).
- $a_{ij}^{\text{post}}(t)$: benefit paid at the *end* of period $[t, t+1)$ conditional on the transition $I_t = i \to I_{t+1} = j$.

The *expected cash flow* conditional on being in state i_0 at time s is

$$\mathbb{E}[CF(t) \cdot \mathbf{1}_{X_t=j} \mid I_{X_s} = i_0] = p_{i_0 j}(s,t)\left[a_j^{\text{pre}}(t) + \sum_k p_{jk}(t)\, a_{jk}^{\text{post}}(t) \cdot v_i(t)\right], \tag{2}$$

where $v_i(t)$ is the one-period discount factor in state i at time t. Summing over j gives the total expected (discounted) cash flow at t.

We note that the to total expected cash flow at time t can be calculated as the sum of the above as follows:

$$\mathbb{E}[CF(t) \mid I_{X_s} = i_0] = \sum_j \left(p_{i_0 j}(s,t)\left[a_j^{\text{pre}}(t) + \sum_k p_{jk}(t)\, a_{jk}^{\text{post}}(t) \cdot v_i(t)\right]\right), \tag{3}$$

2.4 Thiele's Difference Equation

The *prospective reserve* (mathematical provision) in state i at time t is the expected present value of all future net obligations:

$$V_i(t) = \mathbb{E}\Big[\sum_{\tau \geq t} (\text{discounted net benefit at } \tau) \;\Big|\; I_t = i\Big].$$

Theorem 2.3 (Thiele's difference equation, discrete time – **[Koller, Ch. 4]**). The reserves satisfy the backward recursion: $V_i(T) = 0$ for all i (terminal boundary condition at the end of the contract), and for $t = T-1, T-2, \ldots, s$:

$$\boxed{V_i(t) = a_i^{\text{pre}}(t) + \sum_{j \in \mathcal{S}} p_{ij}(t)\, v_i(t) \left[a_{ij}^{\text{post}}(t) + V_j(t+1)\right].} \tag{4}$$

Remark 2.4. The boundary condition implemented in `markovlib` is $V_i(\texttt{start_time}) = 0$ for all i, corresponding to $T = \texttt{start_time}$ (typically $\omega + 1$, where ω is the maximum age in the mortality table).

Equation (4) is the exact recursion implemented in `MARKOVLV._compute_dk`. The key steps are:

1. Initialise $V_i(T) = 0$.
2. Loop $t = T - 1$ down to `stop_time`:
3. For each state i: accumulate $a_i^{\text{pre}}(t)$ plus the weighted sum $\sum_j p_{ij}(t)\, v_i(t)\, [a_{ij}^{\text{post}}(t) + V_j(t+1)]$.

2.5 Technical Premium Decomposition

The reserve can be decomposed (see **[Koller, Ch. 5]**) as:

$$\text{Savings premium:} \quad \text{SP}_i(t) = v_i(t)\, V_{\sigma(i)}(t+1) - V_i(t), \tag{5}$$

$$\text{Risk premium:} \quad \text{RP}_i(t) = \sum_{j \neq \sigma(i)} v_i(t)\, p_{ij}(t) \left[V_j(t+1) - V_{\sigma(i)}(t+1) + a_{ij}^{\text{post}}(t)\right], \tag{6}$$

$$\text{Regular payment:} \quad \text{RegP}_i(t) = a_i^{\text{pre}}(t) + a_{i,\sigma(i)}^{\text{post}}(t), \tag{7}$$

where $\sigma(i)$ is the designated *successor state* of state i (set via `lSetFolgezustand`).

2.6 Mortality Projection

For mortality improvement modelling, base mortality q_x is projected to calendar year y using the exponential (Lee–Carter type) formula:

$$q_x(y) = q_x \cdot \exp\bigl(f_x \cdot (y - y_0)\bigr), \tag{8}$$

where $f_x \leq 0$ is the improvement factor at age x and y_0 is the base year of the table. The interpretation of f_x is a follows: the mortality of a subsequent year follows $q(t+1) = \exp(f) \times q(t)$. Given $\exp(f) = 1 + f + O(f^2)$, hence $f_x = -0.02$ means an annual 2% reduction of the mortality. This is implemented in `ANNUITYLV.dGetQx`, `CAPITALLV.dGetQx`, etc.

3 Library Architecture

3.1 Overview

The class hierarchy is:

Class	Inherits	Purpose
`TABLESERVER`	–	Mortality table wrapper
`MARKOVLV`	–	Core Markov engine (Thiele, Chapman–Kolmogorov)
`CAPITALLV`	`MARKOVLV`	2-state endowment / term-life
`ANNUITYLV`	`MARKOVLV`	3-state single-life annuity
`ANNUITYLV2`	`MARKOVLV`	4-state two-life annuity
`WIDDOWLV`	`MARKOVLV`	Widow pension
`GLMOD`	`MARKOVLV`	402-state group-life portfolio
`ANNMOD`	`MARKOVLV`	1702-state annuity portfolio

Runtime configuration is loaded from `config.json` at import time; mortality tables are loaded from `tables.json`.

3.2 Configuration: `config.json`

`config.json` controls all default parameters. It is loaded once at import time into the module-level constant `CFG`.

Key path	Default	Meaning
`defaults.max_times`	200	Default time horizon
`defaults.max_states`	40	Default state count
`defaults.nr_moments`	1	Default number of moments
`defaults.stochastic_interest`	`false`	Stochastic interest model
`limits.max_moments`	10	Absolute moment ceiling
`limits.tech_eps`	10^{-10}	Numerical zero threshold
`features.with_risk_premium`	`true`	Enable RP/SP/RegP
`features.with_simulation`	`true`	Enable Monte Carlo
`simulation.default_seed`	1234567	Default RNG seed
`tables.tables_file`	`tables.json`	Mortality table file path
`tables.default_base_year`	2000	Default base year y_0
`tables.default_actual_year`	2000	Default valuation year
`tables.default_max_proj`	10000	Max improvement projection years

3.3 Mortality Tables: `tables.json`

`tables.json` contains 157 actuarial mortality tables extracted from the original C++ source. Each entry is keyed by a string identifier and contains the fields listed below.

Field	Type	Description
`id`	string	Unique table identifier, e.g. `"CH-QX-ERF-1995"`
`type`	string	`"qx"` (mortality) or `"fx"` (improvement factor)
`x0`	int	Minimum age in the table
`x_omega`	int	Maximum age ω
`t0`	int	Base calendar year of the table
`i_tech`	float	Technical interest rate (e.g. 0.03)
`gender`	int	0 = male, 1 = female
`qx`	list[float]	Mortality rates, indexed from age 0 to ω

Tables covering ages $[x_0, \omega]$ are zero-padded at the start (ages $0, \ldots, x_0 - 1$). Values $\geq$ 0.99999999 are clamped to 1.0 (consistent with the original C++ `TABLESERVER::dGetValue`).

Tables span actuarial experience from Switzerland (CH), Germany (D/DE), France (F), the United Kingdom (GB/UK), Belgium (B), the USA, Australia, Scandinavia, and several Asian markets.

4 Module Reference

4.1 `TABLESERVER` — Mortality Table Wrapper

4.1.1 Purpose

Provides age-indexed access to a single mortality table from the shared registry, with optional proportional level scaling.

4.1.2 Constructor

```
ts = TABLESERVER()
```

Listing 1: TABLESERVER instantiation

4.1.3 Methods

Method	Description
`vSetTable(pc_table)`	Load a table by ID. The ID may optionally include a scaling factor: `"CH-QX-ERF-1995;0.90"` scales all q_x by 0.9.
`dGetValue(age)`	Return q_x for `age`, applying the level factor. Returns 0 for `age` $< x_0$ and 1 for `age` $> \omega$.
`iTableNumber()`	Numeric index in the registry.
`iX0()`	Minimum age x_0.
`iXOmega()`	Maximum age ω.
`iT0()`	Base calendar year y_0.
`dITech()`	Technical interest rate.

`iGender()`	0 = male, 1 = female.
`pcAllTarifs()`	Newline-separated list of all registered table IDs.

4.1.4 Module-level functions

```
register_table(table_id, x0, x_omega, t0, i_tech, gender, qx)
list_tables()   # returns sorted list of all table IDs
```

Listing 2: Extending the table registry at runtime

4.2 `MARKOVLV` — Core Markov Engine

4.2.1 Purpose

Implements Thiele's difference equation (4), the Chapman–Kolmogorov forward pass for expected cash flows, technical premium decomposition, and Monte Carlo simulation.

4.2.2 Constructor

```
m = MARKOVLV(max_times=200, max_states=40, nr_moments=1)
```

Listing 3: MARKOVLV constructor

4.2.3 Configuration setters

Method	Description
`vReset()`	Clear all Pre/Post/Pij/Disc data.
`vSetStartTime(t)`	Set T (backward recursion starts here).
`vSetStopTime(t)`	Set s (backward recursion ends here).
`vSetNrStates(n)`	Set the number of states $\lvert\mathcal{S}\rvert$.
`vSetGetData(status)`	If `True`: `dSet*` methods act as getters.
`vSetInterestModel(stoc)`	If `True`: stochastic interest (discount depends on target state).
`vSetDefaultNrMoments(n)`	Number of moments to compute.
`lSetFolgezustand(i, j)`	Set successor state $\sigma(i) = j$ (for RP/SP decomposition).

4.2.4 Input setters

Method	Description
`dSetPre(t, from_s, to_s, v)`	Set $a^{\text{pre}}_{from_s}(t) = v$. The `to_s` argument is ignored.
`dSetPost(t, from_s, to_s, v)`	Set $a^{\text{post}}_{from_s,to_s}(t) = v$.
`dSetPij(t, from_s, to_s, v)`	Set $p_{from_s,to_s}(t) = v$.

`dSetDisc(t, from_s, to_s, v)`	Set $v_{from_s}(t) = v$ (diagonal: `from_s == to_s` for deterministic interest).

If `dAddBenefits = True` (set in subclasses like `GLMOD`), `dSetPre` and `dSetPost` *add* to existing values rather than overwriting — enabling superposition of multiple policies on the same state.

4.2.5 Result getters

Method	Description
`dGetDK(t, state, moment=1)`	Reserve $V_{\texttt{state}}^{(\texttt{moment})}(t)$; equation (4).
`dGetCF(t, init_state, time_state)`	$\mathbb{E}[CF(t) \cdot \mathbf{1}_{I_t=\texttt{time_state}} \mid I_s = \texttt{init_state}]$; equation (3).
`dGetRP(t, state)`	Risk premium; equation (6).
`dGetSP(t, state)`	Savings premium; equation (5).
`dGetRegP(t, state)`	Regular payment stream; equation (7).
`lGetMaxTime()`	Value of `max_times`.
`lGetNrStates()`	Current number of states.
`lGetStartTime()`	T (start of backward recursion).
`lGetStopTime()`	s (end of backward recursion).

4.2.6 Simulation

Method	Description
`vSetInitState(state)`	Initial state for trajectory simulation.
`vNewSeed(seed)`	Reseed the random number generator.
`vGenerateTrajectory()`	Simulate one Markov path forward.
`vGetState(t)`	State along the simulated path at time t.
`dGetRandCF(t)`	Cash flow along the simulated path at time t.
`dGetMeanCF(t, state, nr_sim)`	Monte Carlo mean CF at $(t, \texttt{state})$ over `nr_sim` paths.
`dGetMeanDK(t, state, nr_sim)`	Monte Carlo mean reserve.
`vResetMeanResults()`	Clear accumulated simulation results.

4.3 `CAPITALLV` — Endowment / Term-Life

4.3.1 Purpose

Two-state model (alive = 0, dead = 1) for endowment and term-life insurance with mortality projection.

4.3.2 State space

$\mathcal{S} = \{0 = \text{alive},\ 1 = \text{dead}\}$. Transitions: $p_{00}(t) = 1 - q_x(t, y)$; $p_{01}(t) = q_x(t, y)$; $p_{11}(t) = 1$ (absorbing).

4.3.3 Key methods

Method	Description
`iSetTable(table_id)`	Load mortality table; sets q_x, discount, start time.
`vSetStartTime(t)` / `vSetStopTime(t)`	Contract end age / entry age.
`vSetDeath(value)`	Constant death benefit for all ages.
`vSetSurvival(t, value)`	Survival benefit paid at maturity time t. Note: stored at index $t-1$ (C++ convention).
`vSetPremium(value)`	Annual premium (constant); set as negative pre-benefit.
`vSetDeathGen(t, v)` / `vSetSurvivalGen(t, v)` / `vSetPremiumGen(t, v)`	Override individual time points.
`dSetQx(t, v)`	Set base mortality at age t.
`dSetFx(t, v)`	Set improvement factor at age t.
`dSetDisc(t, v)`	Set discount factor at absolute time t.
`dSetBaseYear(t)` / `dSetActualYear(t)`	Set y_0 and valuation year y for equation (8).
`dGetDK(t)`	Reserve $V_0(t)$ (alive state).
`dGetCF(t)`	Expected cash flow at time t.
`dGetQx(t, year)`	Projected $q_x(t, \texttt{year})$; equation (8).
`dGetEV(t)`	Simplified embedded value at time t.

4.4 `ANNUITYLV` — Single-Life Annuity

4.4.1 Purpose

Three-state annuity model with optional guarantee period and time-varying benefit schedule.

4.4.2 State space

- **0** = alive (no current guarantee)
- **1** = alive within guarantee period
- **2** = dead outside guarantee (absorbing)

Transitions at time t (with q_t the projected mortality):

$$p_{00}(t) = 1 - q_t, \qquad p_{0j}(t) = q_t \ \text{ where } j = \begin{cases} 1 & s \le t < s+g \\ 2 & \text{otherwise} \end{cases},$$

$$p_{11}(t) = 1, \quad p_{22}(t) = 1.$$

Benefits: $a_i^{\text{pre}}(t) = b(t) \cdot d_{\text{pre}}$ and $a_{ii}^{\text{post}}(t) = b(t) \cdot d_{\text{post}}$ for $t \ge s$, where $b(t)$ is the benefit schedule and $d_{\text{pre}} + d_{\text{post}} = 1$.

4.4.3 Key methods

Method	Description
`iSetTable(table_id)`	Load table; sets q_x, discount, start time.
`vSetSAge(t)`	Start age s of annuity payment.
`vSetG(g)`	Guarantee period length g.
`dSetPre(d_pre)`	Fraction paid at start of period ($d_{\text{pre}} \in [0, 1]$).
`vLeistReset()`	Reset benefit to 1 for all ages.
`vSetLeistLinear(t0, t1, start, incr)`	$b(t) = \texttt{start} + k \cdot \texttt{incr}$ for $t \in [t_0, t_1]$.
`vSetLeistExp(t0, t1, start, factor)`	$b(t) = \texttt{start} \cdot \texttt{factor}^k$ for $t \in [t_0, t_1]$.
`dSetRelativeQxForTime(t, v)`	Multiply q_x at projection step t by v.
`dGetDK(t)`	$V_0(t)$.
`dGetCF(t)`	Expected cash flow at time t.
`dGetTpx(k)`	k-year survival probability ${}_kp_x$.
`dGetTqx(k)`	k-year mortality probability ${}_kq_x$.

4.5 `ANNUITYLV2` — Two-Life Annuity

4.5.1 Purpose

Four-state model for joint-life and last-survivor annuities with two separate mortality tables.

4.5.2 State space

- **0** = both x and y alive
- **1** = x dead, y alive (y-annuity may pay)
- **2** = x alive, y dead (x-annuity may pay)
- **3** = both dead (absorbing)

Transitions: $p_{00} = (1 - q_x)(1 - q_y)$, $p_{01} = (1 - q_x)\, q_y$, $p_{02} = q_x\,(1 - q_y)$, $p_{03} = q_x\, q_y$. State 1: $p_{11} = 1 - q_x$, $p_{13} = q_x$. State 2: $p_{22} = 1 - q_y$, $p_{23} = q_y$.

4.5.3 Key methods

Method	Description
`iSetTable1(id)` / `iSetTable2(id)`	Load mortality tables for lives x and y.
`vSetSAge1(t)` / `vSetSAge2(t)`	Payment start ages for each life.
`dSetY_Minus_X(y, x)`	Age gap $y - x$; stored as `_y_minus_x`.
`dSetBenefit(state, value)`	Annual benefit while in `state` (states 0, 1, 2).
`vSetLeistLinear` / `vSetLeistExp`	Set time-varying benefit schedule.
`dGetDK(t, state=0)`	$V_{\texttt{state}}(t)$.

`dGetCF(t)`	Total expected cash flow at t.

4.6 `WIDDOWLV` — Widow Pension

4.6.1 Purpose

Two-state model for the insured (alive / dead). Upon death, the EPV of the widow's (or widower's) annuity is computed using an embedded `ANNUITYLV` sub-object and deposited as a post-transition benefit.

4.6.2 State space

$\mathcal{S} = \{0 = \text{insured alive},\ 1 = \text{insured dead}\}$. Transition to state 1 occurs with probability $q_x(t) \cdot h_x(t)$, where $h_x(t) \in [0, 1]$ is the probability that a widow exists. State 1 is absorbing.

4.6.3 Key methods

Method	Description
`dSetQx(t, v)`	Mortality of the insured.
`dSetQy(t, v)`	Mortality of the survivor (forwarded to internal `ANNUITYLV`).
`dSetFx(t, v)` / `dSetFy(t, v)`	Improvement factors for insured / survivor.
`dSetHx(t, v)`	Probability of a surviving spouse at death time t.
`dSetYx(t, v)`	Age of survivor when insured dies at time t.
`dSetDisc(t, v)`	Set discount (negative t fills all 2500 positions).
`vLeistReset` / `vSetLeistLinear` / `vSetLeistExp`	Survivor's annuity benefit schedule.
`dGetDK(t)`	$V_0(t)$ (reserve for insured alive).
`dGetCF(t)`	Expected cash flow at t.

4.7 `GLMOD` — Group Life Portfolio (402 states)

4.7.1 Purpose

Aggregate portfolio model embedding hundreds of individual policies in a single 402-state Markov chain. Each entry age and product type occupies its own state; `vUpdateOperator` fills all transition probabilities from second-order mortality.

4.7.2 State layout

States	Block	Contents
0–99	`MAN+CAPITAL+x`	Male endowment, entry age x
100–199	`WOMAN+CAPITAL+x`	Female endowment, entry age x
200–299	`MAN+ANNUITY+x`	Male annuity, entry age x
300–399	`WOMAN+ANNUITY+x`	Female annuity, entry age x
400	`SURRENDER`	Absorbing surrender state
401	`DEATH`	Absorbing death state

4.7.3 Key methods

Method	Description
`dSetQx(tbl, typ, sex, age, v)`	Set q_x; tbl 0=KTAF/1=RTAF, typ 0=2nd/1=1st order, sex 0/1.
`dSetFx(tbl, typ, sex, age, v)`	Set improvement factor.
`dSetSx(tbl, typ, sex, age, v)`	Set lapse rate (single vector, args ignored).
`dSetBaseYear(tbl, typ, sex, y)`	Set y_0 for the given table/type/sex.
`dSetActualYear(y)`	Set valuation year.
`dSetDisc(t, v)`	Set discount factor at time t.
`vStress(type, amount)`	Apply scenario stress (types 0=reset, 1=qx, 2=fx, 3=sx, 4=yield).
`vSetRKWAnnuity(type, amount)`	Surrender value scaling for annuities.
`vSetRKWEndowment(type, amount)`	Surrender value scaling for endowments.
`vAddAnnuity(sex, x, s, n, leist, praem, itech)`	Add single-life annuity policy to portfolio.
`vAddEndowment(sex, x, s, leist, praem, itech)`	Add endowment policy.
`vAddWiddow(sex, x, s, n, leist, praem, itech)`	Add widow pension rider.
`vUpdateOperator()`	Build all Pij from mortality and lapse; must be called before `dGetDK`.
`dGetDK(t)`	Aggregate portfolio reserve $\sum_i V_i(t)$.
`dGetDKDetail(t, state)`	Reserve for a single state.
`dGetDKTilde(t)`	Forward-accumulated DK at time t.
`dGetStatDK(type)`	Static (technical) DK accumulated during `vAdd*` calls.
`dGetFVDK()`	FV (market discount) DK from `vAdd*` calls.
`dGetCF(t)`	Aggregate expected cash flow.
`dGetCFDetail(t, state)`	Cash flow for one state.
`iReadInforce(iP, iL, filename)`	Read a portfolio inforce file.

`vPrintTex(filename)`	Write LaTeX reserve table.

4.8 ANNMOD — Annuity Portfolio (1702 states)

4.8.1 Purpose

Large-scale annuity portfolio supporting 1-life, joint-life, last-survivor, and widow annuities across a 5-band age-gap dimension.

4.8.2 State layout

State range	Contents
0–99	Male 1-life annuities (entry age x)
100–199	Female 1-life annuities (entry age x)
200–299	2-life m/m, band 0 ($\Delta \leq -7$)
300–399	2-life m/f, band 0
400–499	2-life f/f, band 0
500–799	Bands 1 ($-7 < \Delta \leq 0$): m/m, m/f, f/f
800–1099	Bands 2–4 similarly
1700	Zinsrente (pure interest, no mortality)
1701	Death (absorbing)

Here $\Delta = x - y$ is the age difference between the two lives.

4.8.3 Key methods

Method	Description
`dSetQx/Fx/Sx`	As in `GLMOD`.
`vAddAnnuity1(sex, x, s, n, leist, praem, itech, incr)`	Single-life annuity with benefit escalation `incr`.
`vAddAnnuity0(x, s, n, leist, praem, itech, incr)`	Pure interest (Zinsrente) annuity.
`vAddAnnuity2xy(...)`	Joint-life (beide leben) annuity.
`vAddAnnuity2xyBar(...)`	Last-survivor annuity (= ann1(x)+ann1(y)− ann2xy).
`vAddAnnuity2xToy(...)`	Widow annuity for y after death of x.
`vAddAnnuity2yTox(...)`	Widower annuity for x after death of y.
`vUpdateOperator()`	Fill all Pij; must be called before `dGetDK`.
`dGetDK(t)`	$\sum_i V_i(t)$.
`dGetCF(t)`	Aggregate cash flow.

5 Examples

5.1 Example 1: Simple 2-State Model (Manual Setup)

This example demonstrates the raw `MARKOVLV` interface: a 5-year term contract with constant mortality $q = 0.01$ and interest $i = 3\%$. The reserve at $t = 0$ should equal the EPV of a CHF 100 000 death benefit minus the EPV of annual premiums of CHF 500.

```
import markovlib as ml

q    = 0.01    # annual mortality
i    = 0.03    # technical interest
disc = 1.0 / (1.0 + i)
n    = 5       # term in years

m = ml.MARKOVLV(max_times=10, max_states=2, nr_moments=1)
m.vSetNrStates(2)
m.vSetStopTime(0)        # entry age (relative time)
m.vSetStartTime(n)       # contract end

for t in range(n):
    m.dSetPij(t, 0, 0, 1.0 - q)   # alive -> alive
    m.dSetPij(t, 0, 1, q)          # alive -> dead
    m.dSetPij(t, 1, 1, 1.0)        # dead absorbing
    m.dSetDisc(t, 0, 0, disc)
    m.dSetDisc(t, 1, 1, disc)
    m.dSetPre(t,  0, 0, -500.0)    # premium (negative = outflow)
    m.dSetPost(t, 0, 1, 100_000)   # death benefit

V0 = m.dGetDK(0, 0, 1)            # reserve at t=0, alive state
print(f"Reserve at issue: {V0:,.2f}")
# Expected: EPV(benefits) - EPV(premiums)
```

Listing 4: 2-state MARKOVLV example

5.2 Example 2: Single-Life Annuity from Table

Load a Swiss annuity table and compute the EPV of a unit annuity for a 65-year-old female with a 10-year guarantee period.

```
import markovlib as ml

ann = ml.ANNUITYLV()
ann.iSetTable("CH-QX-ERF-1995")  # Swiss female annuity table 1995
ann.vSetStopTime(65)              # valuation age
ann.vSetSAge(65)                  # payments start at 65
ann.vSetG(10)                     # 10-year guarantee period

# Fill discount at 3% across all positions (table sets its tech rate,
# but we can override)
for t in range(2500):
    ann.dSetDisc(t, 1.0 / 1.03)

dk = ann.dGetDK(65)
print(f"EPV of unit annuity at age 65: {dk:.4f}")

# Expected cash flows for first 5 years
```

```
for t in range(65, 70):
    print(f"  CF({t}): {ann.dGetCF(t):.6f}")
```

Listing 5: ANNUITYLV from mortality table

5.3 Example 3: Endowment Contract

A 30-year endowment for a 35-year-old male. Death benefit and survival benefit CHF 100 000; annual premium CHF 2 500.

```
import markovlib as ml

cap = ml.CAPITALLV()
cap.iSetTable("CH-QX-EKM-1995")  # Swiss male capital table 1995
cap.vSetStopTime(35)              # entry age
# start_time is set automatically by iSetTable to omega+1

# Set contract term: maturity at 65
cap.vSetStartTime(65)
cap.vSetDeath(100000.0)
cap.vSetSurvival(65, 100000.0)   # survival benefit at maturity
cap.vSetPremium(2500.0)

# Override discount at 2.5%
for t in range(2500):
    cap.dSetDisc(t, 1.0 / 1.025)

V = cap.dGetDK(35)
print(f"Net reserve at age 35 (entry): {V:,.2f}")
print(f"Net reserve at age 50:         {cap.dGetDK(50):,.2f}")
```

Listing 6: CAPITALLV endowment example

5.4 Example 4: Two-Life Last-Survivor Annuity

Male age 65 and female age 62; benefit 1.0 paid while at least one survives (last survivor), and 0.6 for the surviving single life.

```
import markovlib as ml

a2 = ml.ANNUITYLV2()
a2.iSetTable1("CH-QX-ERM-1995")   # male
a2.iSetTable2("CH-QX-ERF-1995")   # female
a2.vSetStopTime(65)                # primary life entry age
a2.vSetStartTime(121)
a2.vSetSAge1(65)                   # payments start at 65 for x
a2.vSetSAge2(65)                   # same for y (relative to y's age)
a2.dSetY_Minus_X(62, 65)           # y is 3 years younger (y-x = -3)
a2.dSetBenefit(0, 1.0)             # both alive: full benefit
a2.dSetBenefit(1, 0.6)             # y alone: 60%
a2.dSetBenefit(2, 0.6)             # x alone: 60%

for t in range(2500):
    a2.dSetDisc(t, 1.0 / 1.03)

print(f"DK (both alive, t=65): {a2.dGetDK(65, 0):.4f}")
print(f"DK (y alone,    t=70): {a2.dGetDK(70, 1):.4f}")
```

```
print(f"DK (x alone,    t=70): {a2.dGetDK(70, 2):.4f}")
```

Listing 7: ANNUITYLV2 joint and last-survivor

5.5 Example 5: Widow Pension

Male insured age 45; female survivor 3 years younger. Probability of surviving spouse: 70%. Unit widow's pension.

```
import markovlib as ml

wid = ml.WIDDOWLV()
wid.vSetStopTime(45)     # insured's entry age
wid.vSetStartTime(100)   # projection horizon

# Set constant mortality and improvement
for age in range(130):
    wid.dSetQx(age, 0.005 + age * 0.0003)    # insured (male)
    wid.dSetQy(age, 0.003 + age * 0.0002)    # survivor (female)
    wid.dSetFx(age, -0.015)
    wid.dSetFy(age, -0.020)
    wid.dSetHx(age, 0.70)                    # 70% chance of widow
    wid.dSetYx(age, float(age - 3))          # widow 3 yrs younger

for t in range(2500):
    wid.dSetDisc(t, 1.0 / 1.03)

print(f"Reserve at entry age 45: {wid.dGetDK(45):.4f}")
print(f"Reserve at age 60:       {wid.dGetDK(60):.4f}")
```

Listing 8: WIDDOWLV widow pension

5.6 Example 7: Adding a Custom Mortality Table

Register a custom projection table and use it with `ANNUITYLV`.

```
import markovlib as ml
import math

# Build a simple Makeham-Gompertz table
def makeham_qx(x, A=0.0007, B=0.00005, c=1.09):
    mu = A + B * c**x
    return 1.0 - math.exp(-mu)

qx_values = [makeham_qx(age) for age in range(121)]

ml.register_table(
    table_id  = "CUSTOM-MAKEHAM-2024",
    x0        = 0,
    x_omega   = 120,
    t0        = 2024,
    i_tech    = 0.02,
    gender    = 0,        # male
    qx        = qx_values,
)

ann = ml.ANNUITYLV()
```

```python
ann.iSetTable("CUSTOM-MAKEHAM-2024")
ann.vSetStopTime(60)
ann.vSetSAge(60)
ann.vSetG(0)

print(f"EPV of unit annuity at 60: {ann.dGetDK(60):.4f}")
print(f"Available tables: {len(ml.list_tables())}")
```

Listing 9: Custom mortality table registration

A Appendix A: `config.json` — Example File

Below is the full default `config.json`. Copy this file into the same directory as `markovlib.py` to activate customised defaults.

```json
{
  "_comment": "markovlib configuration - mirrors C++ preprocessor defines",

  "defaults": {
    "max_times":          200,
    "max_states":          40,
    "nr_moments":           1,
    "stochastic_interest": false
  },

  "limits": {
    "max_moments": 10,
    "tech_eps":    1e-10
  },

  "features": {
    "with_risk_premium":           true,
    "allocate_cf_to_exact_times":  true,
    "with_simulation":             true
  },

  "simulation": {
    "default_seed": 1234567
  },

  "tables": {
    "tables_file":         "tables.json",
    "default_base_year":   2000,
    "default_actual_year": 2000,
    "default_max_proj":    10000
  },

  "tex": {
    "column_format": "lrrrrrrrr"
  }
}
```

Listing 10: config.json — full example

Minimal working example

```python
# Place config.json in the same folder as markovlib.py
# with "simulation": {"default_seed": 99999}
# Then:
import markovlib as ml
print(ml.DEFAULT_SEED)    # prints 99999
```

Listing 11: Using config.json with custom seed

B Appendix B: `tables.json` — Structure and Example

File structure

`tables.json` is a JSON object where each key is a table identifier string and the value is a record with the fields described in Section 3.3. The file is loaded once at module import time into the module-level dict `_TABLE_REGISTRY`.

Table identifier naming convention

Identifiers follow the pattern `CC-TYPE-NAME-YEAR`:

- `CC`: country code (CH, D, F, GB, US, ...)
- `TYPE`: `QX` (mortality) or `FX` (improvement)
- `NAME`: insured group (ERM = Swiss male renters, ERF = female, etc.)
- `YEAR`: experience year

Minimal example entry

The entry for `CH-QX-ERF-1995` (Swiss female annuity table 1995):

```json
{
  "CH-QX-ERF-1995": {
    "id":      "CH-QX-ERF-1995",
    "type":    "qx",
    "x0":      0,
    "x_omega": 120,
    "t0":      1995,
    "i_tech":  0.03,
    "gender":  1,
    "qx": [
      0.001562, 0.001218, 0.000933, 0.000701, 0.000519,
      0.000385, 0.000295, 0.000250, 0.000246, 0.000272,
      0.000316, 0.000367, 0.000414, 0.000452, 0.000530,
      ...
      1.0
    ]
  }
}
```

Listing 12: tables.json — single table entry (abbreviated)

Short usage example

```python
import markovlib as ml
import json

# Print metadata for the first 5 tables
for name in ml.list_tables()[:5]:
    entry = ml._TABLE_REGISTRY[name]
    print(f"{name:30s}  x0={entry['x0']:3d}  "
          f"omega={entry['x_omega']:3d}  "
          f"gender={'F' if entry['gender'] else 'M'}  "
          f"i_tech={entry['i_tech']:.4f}")

# Access via TABLESERVER
ts = ml.TABLESERVER()
ts.vSetTable("CH-QX-ERF-1995")
print(f"\nqx at age 65: {ts.dGetValue(65):.6f}")
print(f"qx at age 90: {ts.dGetValue(90):.6f}")
print(f"qx at age 121 (>omega): {ts.dGetValue(121):.1f}")  # returns 1.0

# Level scaling: mortality at 80% of table values
ts.vSetTable("CH-QX-ERF-1995;0.80")
print(f"qx at age 65 (scaled): {ts.dGetValue(65):.6f}")

# Add a custom table with 3 age points (toy example)
ml.register_table(
    table_id="DEMO-QX-2024",
    x0=20, x_omega=22, t0=2024,
    i_tech=0.025, gender=0,
    qx=[0.0] * 20 + [0.001, 0.002, 0.003]  # ages 0-22
)
ts2 = ml.TABLESERVER()
ts2.vSetTable("DEMO-QX-2024")
print(f"Custom table, age 21: {ts2.dGetValue(21):.4f}")
```

Listing 13: Reading a table entry directly

C Test Results

```
a) High Level Tests
===================

  Per-module breakdown:
  Module                  Tests  Passed  Failed   Pass  Status
  ---------------------------------------------------------------
  ANNMOD                    120     120       0  100.0%  OK
  ANNUITYLV                 150     150       0  100.0%  OK
  ANNUITYLV2                120     120       0  100.0%  OK
  CAPITALLV                 120     120       0  100.0%  OK
  GLMOD                     120     120       0  100.0%  OK
  MARKOVLV                  150     150       0  100.0%  OK
  TABLESERVER                90      79      11   87.8%  FAIL
  WIDDOWLV                  120      90      30   75.0%  FAIL
```

```
b) Stochastic Regression Test
markovlib vs markovlv
==============================

SUMMARY
----------------------------------------------------------------------------
  Total scenarios : 165
  Passed          : 165  (100.0%)
  Failed          : 0  (0.0%)
  Overall         : PASS

  Module           Scenarios  Passed  Failed   Pass%  Status
  -------------------------------------------------------
  ANNMOD                  15      15       0  100.0%  OK
  ANNUITYLV               30      30       0  100.0%  OK
  ANNUITYLV2              30      30       0  100.0%  OK
  CAPITALLV               30      30       0  100.0%  OK
  GLMOD                   15      15       0  100.0%  OK
  MARKOVLV                15      15       0  100.0%  OK
  WIDDOWLV                30      30       0  100.0%  OK

c) Other Tests
==============

The generic core routine MARKOVLV() has been tested for DK and
CF by comparing markovlib vs markovlv vs a hand developed version
```

References

[1] M. Koller, *Stochastische Modelle in der Lebensversicherung*, Springer, Berlin, 2000 (2nd edition 2012).

[2] M. Koller, *Stochastic Models in Life Insurance: Selected Topics*, Springer, Berlin, 2000.

[3] H. U. Gerber, *Life Insurance Mathematics*, 3rd edition, Springer, Berlin, 1997.

[4] R. Norberg, "Reserves in life and pension insurance," *Scandinavian Actuarial Journal*, 1991(1), pp. 1–22.

[5] H. Milbrodt and M. Helbig, *Mathematische Methoden der Personenversicherung*, de Gruyter, Berlin, 1999.